

Raport z Projektu 2: Poziomnica i Detektor Przeciżeń

Z płytką na labach były nieźle przeboje, stare kody gryzły się z nowymi, mikrofon był głuchy jak pień, a samo fizyczne kliknięcie przycisku trzęsło układem i włączało alarm przecięcia. Musiałem przenieść buzzer na inny pin (A2), zintegrować cały najważniejszy kod w jednym stabilnym pliku i dopisać sporo zabezpieczeń, ale ostatecznie układ działa bezbłędnie.

1. Architektura i działanie programu

Główna logika kręci się wewnątrz pętli `while(1)`. Odczyty z akcelerometru (MPU6050) idą po I2C i są na bieżąco rzutowane na kąty Roll, Pitch oraz całkowity wektor przecięcia (G).

Żeby zniwelować "skakanie" odczytów, dorzuciłem do kątów prosty filtr dolnoprzepustowy IIR ($0.1 * \text{nowy_odczyt} + 0.9 * \text{stary_odczyt}$), dzięki czemu poziomnica działa płynnie.

Tak samo jak w pierwszym projekcie, nie chciałem, żeby komunikacja blokowała mikrokontroler:

- UART: Odbiór bajtów idzie w tle na przerwaniach do Ring Buffera. Główna pętla wyciąga znaki, składa je w linijki i podaje do parsera.
- ADC (Mikrofon): Czyta sygnał w tle przez DMA (bufor 128 półsłów). Główna pętla wchodzi do akcji tylko wtedy, gdy DMA zgłosi zapętnienie połowy lub całego bufora.

2. Sprzęt i funkcje główne (Wymagania)

Alarm i Buzzer (PWM): Wygenerowanie sygnału wymagało przełączenia buzzera na kanał TIM3_CH2 (fizyczny pin PA4/A2).

- Jeśli przecięcie przekroczy ustawiony próg (np. 1.5g), buzzer wyje ciągłym sygnałem.
- Jeśli zbliża się do progu (przekroczy 80% ustawionej wartości), układ odpala pulsacyjny, ostrzegawczy dźwięk o zmniejszonej do 25% głośności. (*Hack: Wciśnięcie przycisku automatycznie wycisza buzzer na 2 sekundy, żeby siła naciskania na płytkę nie wyzwalała fałszywych alarmów*).

Poziomnica i Kalibracja (Przycisk Niebieski B1): Niebieski przycisk działa na stan niski (GND) i ma własny pomiar czasu:

- Przytrzymanie 5 sekund: Aktualne odchylenie płytki jest zapisywane jako nowy punkt odniesienia (0.00 dla Roll i Pitch). Offsety od razu zapisują się do pamięci Flash.

- Szybki trójklik: (Poniżej 1 sekundy) - zatwierdza i zapisuje maksymalne zanotowane wychylenia układu do Flasha. Wywala znacznik na konsoli.

Mikrofon i 2 kłaśnięcia: Mikrofon miał bardzo słabą czułość, więc w kodzie maksymalnie obniżyłem próg detekcji różnicy napięć (z 2000 na 150). Teraz układ wykrywa 2 kłaśnięcia (albo po prostu 2 puknięcia palcem w moduł mikrofonu w odstępie 150-1000 ms). Po wykryciu układ zapisuje kąty i rzuca na konsolę wielką ramkę: [---> ZNACZNIK: WYKRYTO PODWOJNE KLASNIECIE <---].

3. Komendy UART

Układ reaguje na proste polecenia (zabezpieczyłem parser tak, że w locie zamienia przecinki na kropki, bo wpisywanie ułamków z polskiej klawiatury wcześniej psuło funkcję atof):

- SET_G 1.5 - ustawia próg przeciążenia na 1.5 g (można też wpisać 1,5).
- SET_VOL 50 - ustawia maksymalną głośność alarmu na 50%. *(Domyślnie przy starcie jest 0, układ jest wyciszony, dopóki mu się nie pozwoli na wycie).*
- GET_INFO - wypisuje pełny raport: aktualny Roll i Pitch, zapisane Max Roll i Max Pitch, próg przeciążenia oraz głośność.

4. Pamięć Flash i niezawodność

Wszystkie ważne dane (offsety kalibracji, progi alarmu, głośność i wychylenia maksymalne) połączone są w jedną strukturę i zapisywane do 255. strony w pamięci Flash układu, zabezpieczone "magiczną liczbą" (signature).

Dzięki temu, jeśli układ straci zasilanie (lub wciśnie się czarny przycisk Reset), po ponownym uruchomieniu wczytuje domyślne poprawki, wypisuje BOOT: System Poziomnicy i Alarmu i nadal trzyma idealny "poziom" w miejscu krzywego stołu.